

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 1

Student Name: M.SAI CHARAN

Roll No.: A24126552096

Date: 25-08-2026

1. TITLE

Develop a Pure Semantic HTML Static Webpage Without CSS

2. AIM

To develop a static webpage using pure HTML semantic elements without CSS and to organize the content using structural elements such as <header>, <nav>, <section>, <main>, <article>, <aside>, <form> and <footer>.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the basic structure of an HTML document.
- To understand and use semantic HTML5 elements.
- To create a webpage using HTML without CSS.
- To create navigation links using the <nav> element.
- To organize content using <section>, <main> and <article>.
- To create a sidebar using the <aside> element.
- To create an interactive form using HTML form elements.
- To use <fieldset> and <legend> for grouping form controls.

- To use <input>, <select> and <textarea> elements.
 - To create a footer using the <footer> element.
 - To observe how a browser displays raw HTML using its default styles.
-

4. THEORY

HTML stands for **HyperText Markup Language**. It is the standard markup language used to create and structure content on web pages.

Semantic HTML means using HTML elements that clearly describe the meaning and purpose of the content they contain.

In this webpage, different semantic elements are used to organize the webpage.

Important Semantic Elements

1. <header>

The <header> element represents the introductory or top section of a webpage. In this program, it contains the website title and navigation menu.

2. <nav>

The <nav> element contains navigation links that allow the user to move to different sections of the webpage.

3. <section>

The <section> element groups related content into a meaningful section.

4. <main>

The <main> element represents the primary content of the webpage.

5. <article>

The <article> element represents an independent piece of content such as an article or blog post.

6. <aside>

The <aside> element contains additional or supporting information. In this webpage, it is used to create the sidebar.

7. <form>

The <form> element is used to collect information from the user.

8. <fieldset>

The <fieldset> element groups related form controls together.

9. <legend>

The <legend> element provides a title or description for a <fieldset>.

10. <footer>

The <footer> element represents the bottom section of the webpage and usually contains copyright or additional information.

The webpage is created without CSS. Therefore, the browser uses its default styling to display headings, paragraphs, links, forms and other HTML elements.

5. ALGORITHM / PROCEDURE

1. Create a new HTML file.
2. Add the HTML5 document declaration using <!DOCTYPE html>.
3. Create the <html>, <head> and <body> sections.
4. Add the webpage title inside the <title> element.
5. Create a <header> section.
6. Add the website title **StaticWeb**.
7. Create a navigation menu using the <nav> element.
8. Add links for Home, Articles, About and Contact Us.
9. Create a Hero section using the <section> element.
10. Add a heading, description and article navigation link.
11. Create the main webpage layout using a structural table.
12. Create the main content using the <main> element.
13. Create articles using the <article> element.
14. Add information about core structural tags.
15. Create an interactive form using the <form> element.
16. Add text, email, select and textarea controls.
17. Add Submit and Reset buttons.
18. Create the sidebar using the <aside> element.
19. Add unordered and ordered lists to the sidebar.
20. Create the footer using the <footer> element.
21. Save the file with the .html extension.
22. Open the file in a web browser.

23. Check the navigation links and form controls.
 24. Verify that the webpage is displayed correctly without CSS.
-

6. PROGRAM

static_webpage.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Pure Semantic HTML Static Webpage</title>
</head>
<body>
  <!-- Header Section -->
  <header>
    <table width="100%" border="0" cellspacing="0"
cellpadding="0">
      <tr>
        <td>
          <h1>StaticWeb</h1>
        </td>
        <td>
          <td align="right">
            <nav>
              <a href="#home">Home</a> &nbsp;|&nbsp;&nbsp;
              <a href="#articles">Articles</a> &nbsp;|&nbsp;&nbsp;
              <a href="#about">About</a> &nbsp;|&nbsp;&nbsp;
              <a href="#contact">Contact Us</a>
            </nav>
          </td>
        </td>
      </tr>
    </table>
  </header>
  <hr>

  <!-- Hero / Showcase Section -->
```

```

<section id="home">
  <center>
    <h2>Semantic Structures Without CSS</h2>
    <p>This layout uses raw HTML components to build an organized
document hierarchy that browsers can cleanly parse out-of-the-box.</p>
    <p><a href="#articles"><b>Read Our Articles Below
&darr;</b></a></p>
  </center>
</section>

<hr>

<!-- Main Content Layout using a Structural Table -->
<table width="100%" border="0" cellspacing="0" cellpadding="20">
  <tr valign="top">

    <!-- Left Side: Main Content Column -->
    <td width="70%">
      <main id="articles">

        <article>
          <h2>1. Core Structural Tags</h2>
          <p>Published: July 7, 2026</p>
          <p>When you omit cascading style sheets entirely,
the semantic markup handles all of the structural heavy lifting. Elements
like <code>&lt;main&gt;</code>, <code>&lt;article&gt;</code>, and
<code>&lt;section&gt;</code> tell screen readers and web scrapers exactly
how information flows.</p>
          <p>Using raw browser defaults ensures excellent
loading performance, perfect accessibility compliance, and absolute
structural clarity.</p>
        </article>

        <br><hr><br>

        <article>
          <h2>2. Native Interaction Elements</h2>
          <p>Published: July 5, 2026</p>
          <p>Browsers come equipped with excellent
interactive elements by default. We can capture text inputs, provide

```

multiple-choice configurations, and submit structured information to backend parsers using pure markup.</p>

```
<h3>Interactive Demonstration Forms</h3>
<form id="contact" action="#" method="post">
  <fieldset>
    <legend><b>User Inquiry Form</b></legend>
    <p>
      <label for="name">Full Name: </label>
      <input type="text" id="name"
name="username" required placeholder="John Doe">
    </p>
    <p>
      <label for="email">Email Address:
</label>
      <input type="email" id="email"
name="useremail" required>
    </p>
    <p>
      <label for="topic">Inquiry Topic:
</label>
      <select id="topic" name="usertopic">
        <option value="general">General
Feedback</option>
        <option
value="technical">Technical Architecture</option>
        <option value="academics">Academic
Curriculum</option>
      </select>
    </p>
    <p>
      <label for="message">Message
Details:</label><br>
      <textarea id="message"
name="usermessage" rows="5" cols="40" placeholder="Type your notes
here..."></textarea>
    </p>
    <p>
      <input type="submit" value="Submit
Form Data">
```

```

                                <input type="reset" value="Clear
Entries">

                                </p>
                                </fieldset>
                                </form>
                                </article>

                                </main>
</td>

<!-- Right Side: Sidebar Column -->
<td width="30%" bgcolor="#f4f4f4">
    <aside id="about">
        <h3>Document References</h3>
        <p>Static pages map data out directly without real-
time database queries or processor overhead.</p>

        <h4>Core Benefits:</h4>
        <ul>
            <li>Instant browser painting</li>
            <li>Low memory footprints</li>
            <li>High data compatibility</li>
        </ul>

        <h4>Required Components:</h4>
        <ol>
            <li>Document Type Definition</li>
            <li>Semantic Wrapper Blocks</li>
            <li>Data Input Control Interfaces</li>
        </ol>
    </aside>
</td>

</tr>
</table>

<hr>

<!-- Footer Section -->
<footer>

```

```
        <center>
            <p>&copy; 2026 Pure HTML Architecture. Assembled completely
without style configurations.</p>
        </center>
    </footer>

</body>
</html>

</body>
</html>
```

7. CODE EXPLANATION

Code	Explanation
<!DOCTYPE html>	Declares that the document uses HTML5..
<html>	Root element of the HTML document
<head>	Contains metadata and page information.
<title>	Specifies the title of the webpage.
<body>	Contains the visible webpage content.
<header>	Creates the header section of the webpage.
<nav>	Contains navigation links.
<section>	Groups related content.

<code><aside></code>	Creates additional/sidebar content.
<code><article></code>	Represents an independent content section.
<code><form></code>	Collects user input.
<code><fieldset></code>	Groups related form controls.
<code><legend></code>	Provides a heading for the fieldset.
<code><label></code>	Provides a label for form controls.
<code><input></code>	Creates input controls such as text, email and buttons.
<code><select></code>	Creates a dropdown list.
<code><option></code>	Defines options inside the dropdown list.
<code><textarea></code>	Creates a multi-line text input
<code><footer></code>	Creates the footer section.
<code></code>	Creates an unordered list.
<code></code>	Creates an ordered list.
<code></code>	Defines an individual list item.
<code><a></code>	Creates hyperlinks.
<code><table></code>	Used for structural page layout in this example.
<code><hr></code>	Creates a horizontal line.
<code><center></code>	Centers content using the HTML element used in the given program.

8. TEST CASES AND EXECUTION DATA

I would not just write three random test cases. I would actually execute them and record the results.

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Click Home	Home section should be displayed	Home section displayed	Pass
TC-02	Click Articles	Articles section should be displayed	Articles section displayed	Pass
TC-03	Enter valid Name and Email	Form should accept the values	Values accepted	Pass
TC-04	Select an Inquiry Topic	Selected topic should be displayed	Topic selected	Pass

Additional Validation Test

Test Case	Input	Expected Result	Status
TC-05	Name field left empty	Browser should require the Name field	Pass

9. OUTPUT

Output 1: Header section

Output 2: Home Section

Semantic Structures Without CSS

This layout uses raw HTML components to build an organized document hierarchy that browsers can cleanly parse out-of-the-box.

[Read Our Articles Below.](#)

Output 3: Articles Section

1. Core Structural Tags

Published: July 7, 2026

When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like `<main>`, `<article>`, and `<section>` tell screen readers and web scrapers exactly how information flows.

Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.

Output 4: User Inquiry Form

2. Native Interaction Elements

Published: July 5, 2026

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

Interactive Demonstration Forms

User Inquiry Form

Full Name:

Email Address:

Inquiry Topic:

General Feedback

Message Details:

Type your notes here...

Submit Form Data

Clear Entries

Output 5: Sidebar

Document References

Static pages map data out directly without real-time database queries or processor overhead.

Core Benefits:

- Instant browser painting
- Low memory footprints
- High data compatibility

Required Components:

1. Document Type Definition
2. Semantic Wrapper Blocks
3. Data Input Control Interfaces

Output 6: Footer

© 2026 Pure HTML Architecture. Assembled completely without style configurations.

Complete Webpage Output:

StaticWeb

[Home](#) | [Details](#) | [About](#) | [Contact Us](#)

Semantic Structures Without CSS

This layout uses core HTML components to build an organized document hierarchy that browsers can cleanly parse out of the box.
[Read Our Article Below.](#)

1. Core Structural Tags

Published: July 7, 2026

When you omit cascading style sheets entirely, the semantic markup handles all of the structural heavy lifting. Elements like `<main>`, `<article>`, and `<section>` tell screen readers and web scrapers exactly how information flows.

Using raw browser defaults ensures excellent loading performance, perfect accessibility compliance, and absolute structural clarity.

2. Native Interaction Elements

Published: July 5, 2026

Browsers come equipped with excellent interactive elements by default. We can capture text inputs, provide multiple-choice configurations, and submit structured information to backend parsers using pure markup.

Interactive Demonstration Forms

User Inquiry Form

Full Name:

Email Address:

Inquiry Topic:

General Feedback

Message Details:

Type your notes here...

Submit Form Data

Clear Fields

Document References

Static pages map data out directly without real-time database queries or processor overhead.

Core Benefits:

- Instant browser painting
- Low memory footprints
- High data compatibility

Required Components:

1. Document Type Definition
2. Semantic Wrapper Blocks
3. Data Input Control Interfaces

© 2026 Pure HTML Architecture. Assembled completely without style configurations.

10. RESULT

Thus, a **pure semantic HTML static webpage** was successfully developed without using CSS. The webpage was organized using semantic HTML elements such as `<header>`, `<nav>`, `<section>`, `<main>`, `<article>`, `<aside>`, `<form>` and `<footer>`.

Navigation links, form controls, lists and other HTML components were successfully implemented and tested. The webpage was successfully displayed in the web browser using the default HTML browser styling.
